Lux Display Server: A Z Specification

Formal Model of Frame Lifecycle, Client Management, and Scene State

March 2026

Contents

1 Introduction

Document status. This is a formal subsystem specification of the *legacy single-process display server* — the design where one process owns the socket, the scene state, and the render loop. It is not the shipped three-process Hub/Display runtime the system is migrating toward, and it is not the canonical rewrite-target document. For the rewrite target, start with `target/target.md`; for the current whole-system narrative — including the Hub/Display slice that supersedes this single-process model — see `system.tex`. This spec is retained because its refinement and partition tests (`tests/test_display_refinement.py`, `tests/test_display_partition.py`) still hold the current display code to the model defined here.

The Lux display server is an ImGui render loop that listens on a Unix domain socket and renders scenes. Each frame, it accepts new connections, polls clients for messages, renders the current scene, and flushes interaction events.

This specification models the stateful core:

- Client connection tracking and FrameReader association
- Scene replacement, incremental patching, and clearing
- Message dispatch (scene, update, clear, ping)
- Interaction event generation and broadcast
- The FrameReader streaming buffer

We abstract away ImGui rendering (it is stateless from the protocol's perspective) and focus on the state that the protocol cares about. Element ordering and event queue ordering are abstracted to sets for ProB animatability — seq types cause unbounded enumeration during model checking.

2 Basic Types

$$[FD, ELEMID, SCENEID, RAWBYTES, ACTION]$$

FD models socket file descriptors. *ELEMID* and *SCENEID* are opaque identifiers for elements and scenes. *RAWBYTES* represents a chunk of bytes received from a socket. *ACTION* identifies the action string on a button click.

3 Free Types

$$ZBOOL ::= ztrue \mid zfalse$$

Element kinds supported in Phase 1:

$$ElementKind ::= ekText \mid ekButton \mid ekSeparator \mid ekImage$$

Client message types:

$$ClientMsgType ::= cmScene \mid cmUpdate \mid cmClear \mid cmPing$$

Display message types (sent back to clients):

$$DisplayMsgType ::= dmReady \mid dmAck \mid dmPong \mid dmInteraction$$

4 Global Constants

$$\begin{array}{l} maxClients : \mathbb{N} \\ maxElements : \mathbb{N} \\ maxEvents : \mathbb{N} \\ maxBufSize : \mathbb{N} \\ \hline maxClients = 3 \\ maxElements = 3 \\ maxEvents = 3 \\ maxBufSize = 4 \end{array}$$

5 Display Server State

The server tracks connected clients, the current scene (if any), a set of pending interaction events, and the FrameReader buffer state. All fields are flattened into a single state schema for ProB animatability.

DisplayServer

$$
\begin{array}{l}
clients : \mathbb{P}\ FD \\
readers : \mathbb{P}\ FD \\
hasScene : ZBOOL \\
sceneId : SCENEID \\
elemIds : \mathbb{P}\ ELEMID \\
elemKinds : ELEMID \pfun ElementKind \\
eventQueue : \mathbb{P}\ ELEMID \\
listening : ZBOOL \\
bufSize : \mathbb{N} \\
pendingMsgs : \mathbb{N} \\
\hline
readers = clients \\
\#clients \leq maxClients \\
\#elemIds \leq maxElements \\
\#eventQueue \leq maxEvents \\
bufSize \leq maxBufSize \\
elemIds \subseteq \mathrm{dom}\ elemKinds \\
hasScene = ztrue \Rightarrow \quad eventQueue \subseteq elemIds
\end{array}
$$

The key invariants are:

1. Every client has a FrameReader ($readers = clients$).
2. Client count, element count, event queue, and buffer are bounded.
3. Element kinds are defined for all elements in the scene.
4. Queued events reference elements in the current scene.

6 Initialization

Init

$$
\begin{array}{l}
DisplayServer' \\
\hline
clients' = \emptyset \\
readers' = \emptyset \\
hasScene' = zfalse \\
elemIds' = \emptyset \\
elemKinds' = \emptyset \\
eventQueue' = \emptyset \\
listening' = ztrue \\
bufSize' = 0 \\
pendingMsgs' = 0
\end{array}
$$

7 Operations

7.1 Accept Connection

A new client connects. We allocate a fresh FrameReader for it.

AcceptConnection

$\Delta DisplayServer$
$newClient? : FD$

$listening = ztrue$
$newClient? \notin clients$
$\#clients < maxClients$
$clients' = clients \cup \{newClient?\}$
$readers' = readers \cup \{newClient?\}$
$hasScene' = hasScene$
$sceneId' = sceneId$
$elemIds' = elemIds$
$elemKinds' = elemKinds$
$eventQueue' = eventQueue$
$listening' = listening$
$bufSize' = bufSize$
$pendingMsgs' = pendingMsgs$

7.2 Disconnect Client

A client disconnects (error or clean close).

DisconnectClient

$\Delta DisplayServer$
$deadClient? : FD$

$deadClient? \in clients$
$clients' = clients \setminus \{deadClient?\}$
$readers' = readers \setminus \{deadClient?\}$
$hasScene' = hasScene$
$sceneId' = sceneId$
$elemIds' = elemIds$
$elemKinds' = elemKinds$
$eventQueue' = eventQueue$
$listening' = listening$
$bufSize' = bufSize$
$pendingMsgs' = pendingMsgs$

7.3 Receive Scene

A client sends a new scene, replacing any existing one. An empty element set is a removal, not an install: the scene disappears (`hasScene' = zfalse`) rather than lingering as an empty husk — the Hub blanks a removed scene by re-pushing it with no elements, and the display treats that empty push as the dismissal it is. A non-empty push installs the scene as before. Either way the event queue is cleared, since old element references are stale.

ReceiveScene

$\Delta DisplayServer$
$newSceneId? : SCENEID$
$newElemIds? : \mathbb{P}\ ELEMID$
$newElemKinds? : ELEMID \pfun ElementKind$

$newElemIds? \subseteq \mathrm{dom}\ newElemKinds?$
$\#newElemIds? \leq maxElements$
$(newElemIds? \neq \emptyset \Rightarrow hasScene' = ztrue)$
$(newElemIds? = \emptyset \Rightarrow hasScene' = zfalse)$
$sceneId' = newSceneId?$
$elemIds' = newElemIds?$
$elemKinds' = newElemKinds?$
$eventQueue' = \emptyset$
$clients' = clients$
$readers' = readers$
$listening' = listening$
$bufSize' = bufSize$
$pendingMsgs' = pendingMsgs$

7.4 Clear Scene

A client sends a clear message, removing all content.

ClearScene

$\Delta DisplayServer$

$hasScene' = zfalse$
$sceneId' = sceneId$
$elemIds' = elemIds$
$elemKinds' = elemKinds$
$eventQueue' = \emptyset$
$clients' = clients$
$readers' = readers$
$listening' = listening$
$bufSize' = bufSize$
$pendingMsgs' = pendingMsgs$

7.5 Remove Element (Patch)

An update message removes an element by ID from the current scene.

RemoveElement

$\Delta DisplayServer$
$targetId? : ELEMID$

$hasScene = ztrue$
$targetId? \in elemIds$
$targetId? \notin eventQueue$
$elemIds' = elemIds \setminus \{targetId?\}$
$elemKinds' = \{targetId?\} \ndres elemKinds$
$sceneId' = sceneId$
$hasScene' = hasScene$
$eventQueue' = eventQueue$
$clients' = clients$
$readers' = readers$
$listening' = listening$
$bufSize' = bufSize$
$pendingMsgs' = pendingMsgs$

7.6 Button Click (Interaction)

The display detects a button click. An interaction event is queued for broadcast to all clients.

ButtonClick

$$
\begin{array}{l}
\Delta DisplayServer \\
buttonId? : ELEMID \\
\hline
hasScene = ztrue \\
buttonId? \in elemIds \\
elemKinds\ buttonId? = ekButton \\
\#eventQueue < maxEvents \\
eventQueue' = eventQueue \cup \{buttonId?\} \\
hasScene' = hasScene \\
sceneId' = sceneId \\
elemIds' = elemIds \\
elemKinds' = elemKinds \\
clients' = clients \\
readers' = readers \\
listening' = listening \\
bufSize' = bufSize \\
pendingMsgs' = pendingMsgs
\end{array}
$$

7.7 Flush Events

At the end of each frame, queued events are broadcast to all clients and the queue is emptied.

FlushEvents

$$
\begin{array}{l}
\Delta DisplayServer \\
\hline
eventQueue' = \varnothing \\
hasScene' = hasScene \\
sceneId' = sceneId \\
elemIds' = elemIds \\
elemKinds' = elemKinds \\
clients' = clients \\
readers' = readers \\
listening' = listening \\
bufSize' = bufSize \\
pendingMsgs' = pendingMsgs
\end{array}
$$

7.8 Feed Bytes

Feeding bytes into the FrameReader increases the buffer size.

FeedBytes

$\Delta DisplayServer$
$bytesIn? : \mathbb{N}$

$bytesIn? > 0$
$bytesIn? \leq maxBufSize$
$bufSize + bytesIn? \leq maxBufSize$
$bufSize' = bufSize + bytesIn?$
$pendingMsgs' = pendingMsgs$
$clients' = clients$
$readers' = readers$
$hasScene' = hasScene$
$sceneId' = sceneId$
$elemIds' = elemIds$
$elemKinds' = elemKinds$
$eventQueue' = eventQueue$
$listening' = listening$

7.9 Drain Messages

Draining extracts all complete messages from the FrameReader, reducing the buffer.

DrainMessages

$\Delta DisplayServer$
$drained! : \mathbb{N}$
$bytesConsumed? : \mathbb{N}$

$bytesConsumed? \leq maxBufSize$
$bytesConsumed? \leq bufSize$
$bufSize' = bufSize - bytesConsumed?$
$pendingMsgs' = 0$
$drained! = pendingMsgs + bytesConsumed?$
$clients' = clients$
$readers' = readers$
$hasScene' = hasScene$
$sceneId' = sceneId$
$elemIds' = elemIds$
$elemKinds' = elemKinds$
$eventQueue' = eventQueue$
$listening' = listening$

7.10 Shutdown

The display server shuts down: all clients are disconnected, the scene is cleared, events are flushed.

Shutdown

$\Delta DisplayServer$

$clients' = \varnothing$
$readers' = \varnothing$
$hasScene' = zfalse$
$sceneId' = sceneId$
$elemIds' = elemIds$
$elemKinds' = elemKinds$
$eventQueue' = \varnothing$
$listening' = zfalse$
$bufSize' = bufSize$
$pendingMsgs' = pendingMsgs$

8 System Invariants

The key properties maintained across all operations:

1. **Reader–client bijection**: Every connected client has a FrameReader, and every FrameReader belongs to a connected client ($readers = clients$).
2. **Event validity**: Queued interaction events reference elements that exist in the current scene.
3. **Bounded resources**: Client count, element count, event queue length, and buffer sizes are all bounded.
4. **Scene atomicity**: A new scene completely replaces the old one and clears stale events.
5. **Element kind coverage**: Every element ID in the scene has a corresponding kind in the kind map.